PROJECT REPORT

ROOM STATUS TRACKER

A Live Classroom Status Web Application



Software Engineering Project

Submitted By :-

Sarah Singh 1024240071

Prasuk Jain 1024240007

Mansi Singh 1024240090

Harshil Bansal 1024240040

(Group No.:3X15)

BTECH Third Year, AIML

Faculty Advisor: Mr. Raghav B. Venkataramaiyer

Mid-Semester Evaluation

Department of Computer Science and Engineering

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY, PATIALA

Table of Contents

Section	Title	Page
1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	3
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4–5
2.2	Implementation Details	5
2.2.1	Module 1: Core Functionality	5
2.2.2	Module 2: Secondary Features	6
3	Results and Evaluation	7
3.1	Testing Methodology	7
3.2	Performance Metrics	7–8
3.3	Analysis	8
4	Conclusion	8
4.1	Summary of Work	8
4.2	Key Findings	8
4.3	Future Work and Recommendations	8–9
4.4	Final Remarks	9
	References	9

1. Introduction

1.1 Background and Context

In a college, the timetable tells students what is planned. However, it does not always tell them what is happening in a room right now. A class may be cancelled, shifted, or delayed, or the teacher may not have arrived yet. Shared activity spaces can also become occupied by societies.

The Room Status Tracker is a web application that gives students and staff one place to check the current status of classrooms, Lecture Theatres (LTs), and shared Activity Spaces. Class Representatives and Society Representatives can report changes in the rooms they handle. Teachers and the Department Office of Administration can manage the system at a higher level.

The current milestone is focused on the frontend implementation. The team has developed the user-facing structure, dashboard-oriented screens, room representations, status indicators, navigation, and role-oriented interface elements. The backend, database, authentication service, and live data integration are planned for the next phase. This report therefore documents the work completed so far while also describing the architecture of the intended complete system.

1.1.1 Problem Statement

The main problem is that the timetable may not match the actual use of a room. Students may have to walk to a room to check if a class is running, if the teacher has arrived, or if the room is free. Societies can face the same problem when checking a shared activity space.

The proposed solution is a single platform that shows the latest status of each room. The current frontend provides the interface for this system. The next phase will connect it to the backend so that data can be saved, users can log in, permissions can be checked, and room updates can be shown live.

1.2 Project Objectives

1. Develop a clear interface for viewing the status of CSED classrooms, Lecture Theatres, and shared Activity Spaces.
2. Design role-specific screens for Students, Class Representatives, Society Representatives, Teachers, and Admin/DOA.
3. Provide a consistent visual representation of room status, notes, and update time.
4. Design the application so authorized users can update only the rooms associated with their responsibilities.
5. Establish a complete-system architecture based on React, FastAPI, PostgreSQL, and college-email authentication.
6. Maintain a timestamped history of room-status changes in the planned backend for accountability.

2. Methodology and Design

2.1 System Architecture

The proposed system uses a client-server architecture. The React application provides the user interface. The FastAPI backend will handle the main system logic and security checks. PostgreSQL will store the data, and an email service will be used for OTP or magic-link login. The frontend and backend will communicate using HTTPS and JSON.

Only the frontend has been completed at this stage. Therefore, the architecture diagram separates the completed part from the planned backend and database work. This shows the current project status clearly.

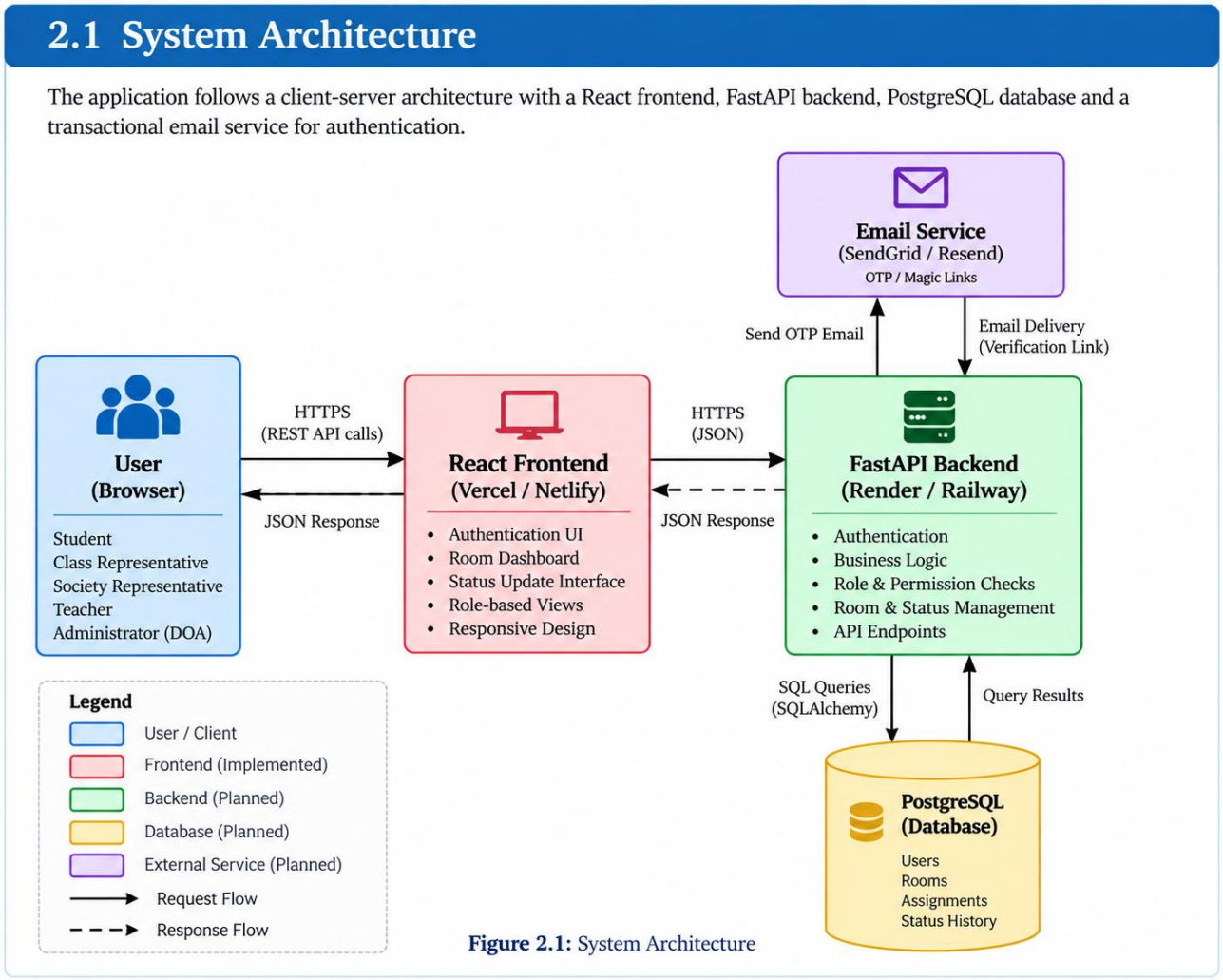


Figure 2.1: System Architecture and Current Development Status

2.1.1 High-Level Design

The interface is built around two main tasks: viewing room information and updating it when a user has permission. The dashboard can group rooms by building or room type. Room cards and status badges keep the display simple and consistent. Separate screens are planned for CRs, Society Representatives, Teachers, and administrators.

In the complete system, the backend will check who can perform each action. A user's role gives general permissions, while room assignments decide which rooms the user can update. Each status change will also have a time and user attached to it. This will keep a history of room changes.

2.1.2 Technical Stack

Layer	Technology	Purpose	Status
Frontend	React	Dashboard, navigation, room cards and role-oriented UI	Implemented
Backend	FastAPI / Python	System logic, login, permissions and room operations	Planned
Database	PostgreSQL	Stores users, rooms, assignments and status history	Planned
ORM	SQLAlchemy	Database interaction in the backend	Planned
Authentication	College Email + OTP / Magic Link	Passwordless user verification	Planned
Email	SendGrid / Resend	OTP or magic-link delivery	Planned
Migrations	Alembic	Manages database schema changes	Planned
Deployment	Vercel/Netlify + Render/Railway	Hosts the frontend and backend	Planned

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The main work completed so far is the frontend for the room-status system. The interface is built around a dashboard that makes room information easy to understand. Room cards, status badges, grouping, navigation, and role-based screens are the main parts of the interface.

The frontend has been designed for the full workflow. Students mainly view information. CRs and Society Representatives need screens for room updates. Teachers and administrators need management screens. At present, these screens are part of the frontend. Actual permissions and saved updates will be handled by the backend.

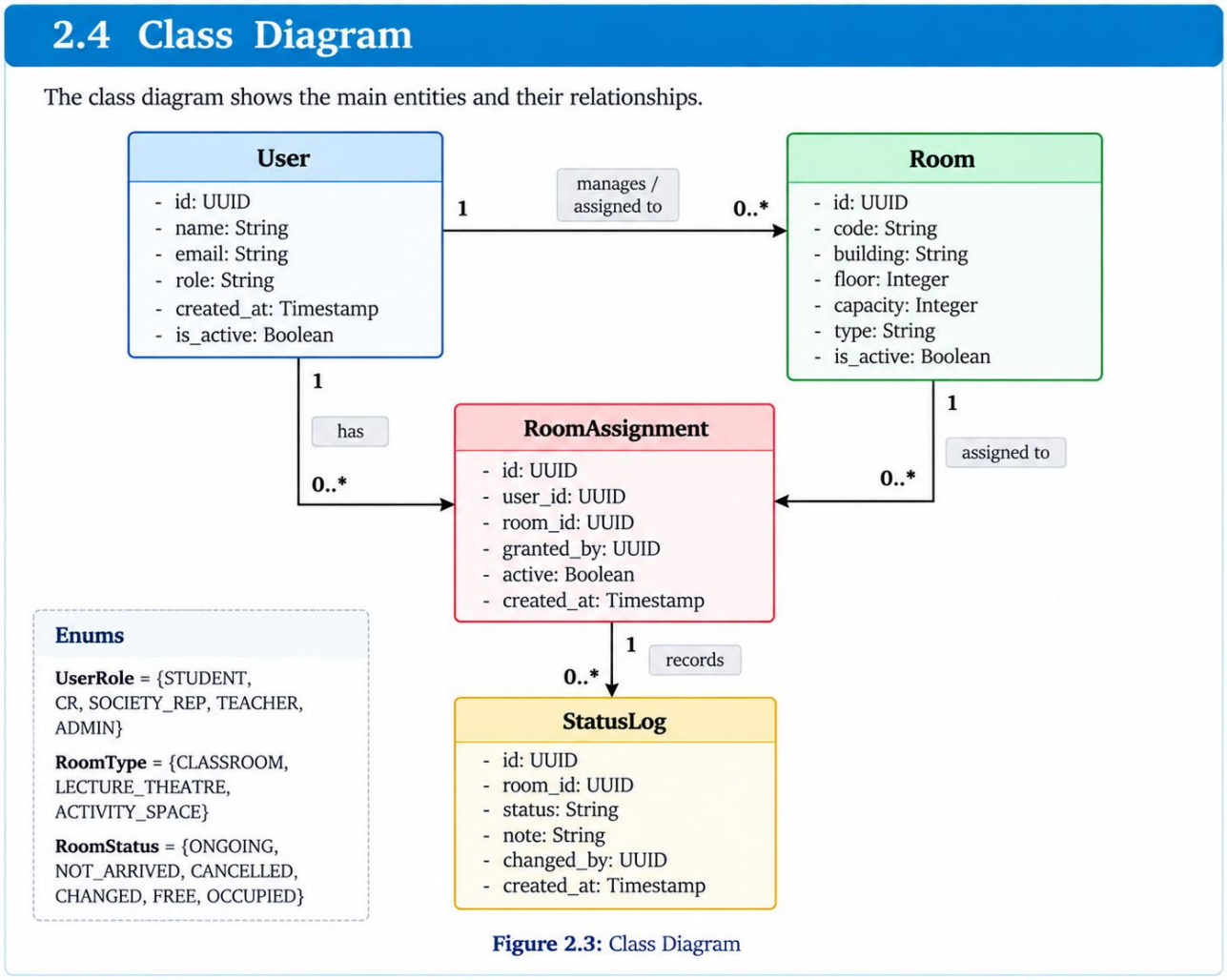


Figure 2.2: UML Class Diagram of the System

2.2.2 Module 2: Secondary Features

Other features include role-based navigation, room grouping, status display, and the planned login and permission system. The room information will look the same for all users, while the available actions will depend on the user's role.

The planned permission model gives students viewing access. CRs can update their assigned classrooms, and Society Representatives can update their assigned Activity Spaces. Teachers can manage CR access for their classes and make suitable overrides. Admin/DOA will have wider management access. These permissions will be checked by the backend.

2.3 Use Case Diagram

The use case diagram shows the main interactions between users and the system.

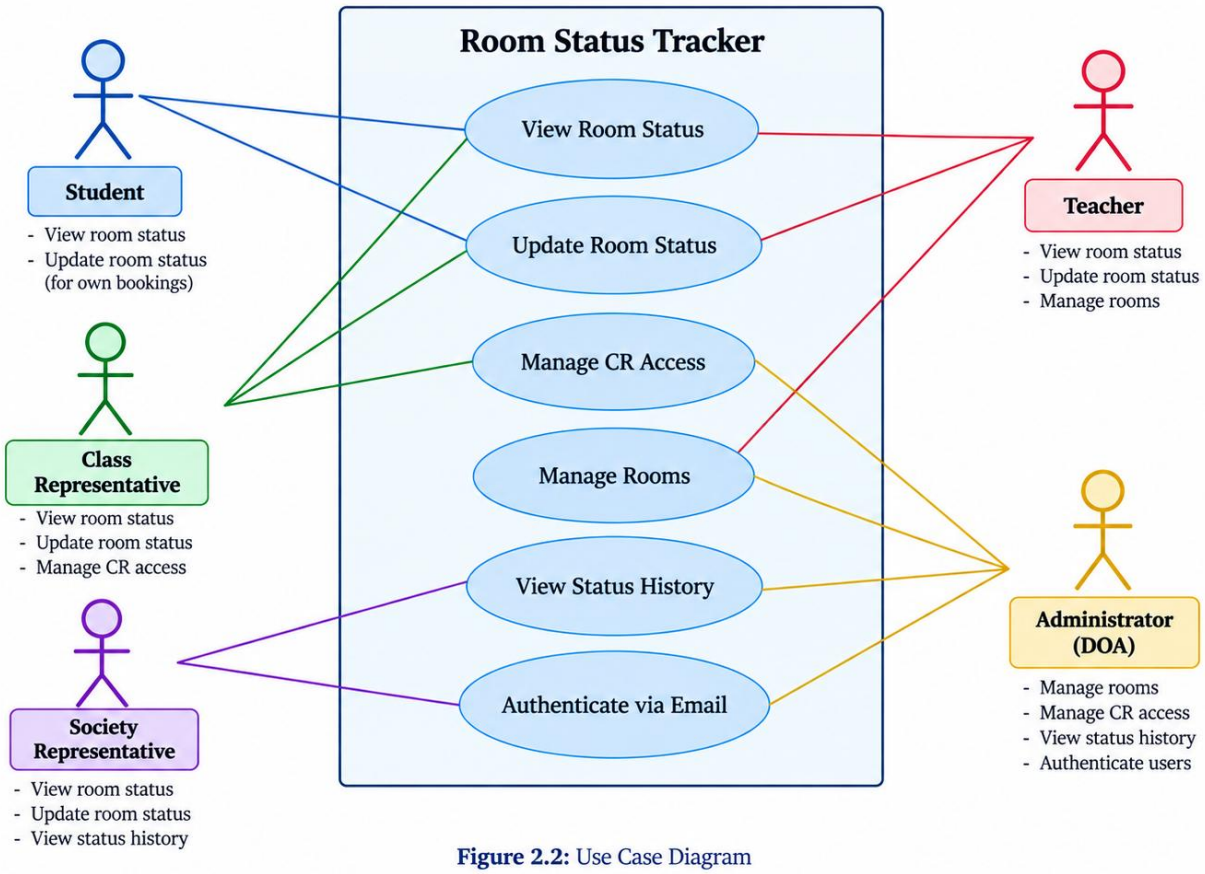


Figure 2.2: Use Case Diagram

Figure 2.3: Use Case Diagram of the System

3. Results and Evaluation

3.1 Testing Methodology

Testing at this stage is focused on the frontend. The team has checked the main screens, navigation, room cards, status indicators, and role-based screens. The checks focus on whether the interface looks correct and responds properly to user actions.

Full testing will be done after the FastAPI backend and PostgreSQL database are connected. It will cover login, permissions, saved data, room-status updates, status history, and multiple users using the system at the same time. User testing can then be done with students, CRs, teachers, and administrators.

3.2 Performance Metrics

Metric	Current Status	Final Evaluation
Navigation and page rendering	Reviewed during frontend development	Measure with complete application
Room-card / status display	Implemented and reviewed	Evaluate with realistic room volume
Authentication latency	Not applicable yet	Measure after backend integration
Database response time	Not applicable yet	Measure after PostgreSQL integration
Concurrent updates	Not applicable yet	Load-test in backend phase
Authorization correctness	UI states designed	Verify server-side after integration

No numerical backend performance values are reported because those components are not yet implemented. Reporting measured values at this stage would not accurately represent the current project work.

3.3 Analysis

The current milestone gives the project a working user-interface foundation. The main workflow has been converted into clear screens and components. The design also leaves a clear path for connecting live data later.

One important design point is permission control. A CR should only update the classroom assigned to them, not every room. In the same way, a Society Representative should only update the Activity Space assigned to them. The planned data model and backend checks are designed for this.

The main limitation is that the current prototype is not connected to the backend, database, or login service yet. Live updates, saved status history, and server-side security will be completed in the next phase.

4. Conclusion

4.1 Summary of Work

The Room Status Tracker is designed to solve a simple problem: helping users find the current status of classrooms and shared spaces. In the current phase, the team has completed the frontend foundation, including the dashboard, room cards, status display, navigation, and role-based screens.

The team has also defined the architecture and data model for the complete application. The planned system uses React, FastAPI, PostgreSQL, college-email authentication, role-based permissions, and status history. The report clearly separates the work already completed from the work planned for the next phase.

4.2 Key Findings

- The current timetable-based approach does not provide sufficient real-time visibility into room conditions.
- A simple dashboard can reduce the need for students to physically check rooms for basic status information.
- Different users require different levels of access, making role- and room-specific permissions important.
- Maintaining status history can improve accountability and trust in the information shown.
- Separating frontend presentation from backend business logic provides a clear path for completing the system.

4.3 Future Work and Recommendations

7. Implement the FastAPI backend and connect it to the existing frontend.
8. Integrate PostgreSQL for users, rooms, assignments, and status history.
9. Implement college-email OTP or magic-link authentication.
10. Enforce role and room-specific permissions on the server.
11. Connect the dashboard to live persistent data and test appropriate refresh or real-time update behaviour.
12. Complete functional, integration, performance, and user testing with realistic usage.
13. Consider notifications, analytics, bulk assignment, and mobile/PWA support as later enhancements.

4.4 Final Remarks

The current milestone gives the project a clear foundation. The frontend shows how users will use the system, while the architecture and UML diagram show how the complete system will be built. The next phase will connect the frontend to the backend and database and turn the prototype into a working live room-status system.

References

1. FastAPI Documentation: Asynchronous Web Framework for Python. Available at: https://fastapi.tiangolo.com/
2. React Documentation: A JavaScript library for building user interfaces. Available at: https://react.dev/
3. PostgreSQL Global Development Group: PostgreSQL Relational Database Management System. Available at: https://www.postgresql.org/

4. SQLAlchemy ORM Documentation: Python SQL Toolkit and Object Relational Mapper. Available at: https://www.sqlalchemy.org/